

Calibración de un Modelo de Crecimiento Poblacional (EcoModelo)

Título completo: Calibración de un Modelo de Crecimiento Poblacional (EcoModelo) mediante Algoritmos de Backtracking y Poda

1. Integrantes

Simon Ospina Gomez, Juan Sebastian Yepez Mazo, Sebastian Pardo Cardona
Universidad Nacional de Colombia (UNAL), Sede Manizales

2. Resumen breve

Este informe describe cómo ajustamos un modelo matemático de crecimiento de población (Eco Modelo) a datos reales recopilados durante cinco días. Queremos encontrar los mejores valores para la tasa de crecimiento (r) y la capacidad de carga (K) de modo que las simulaciones se ajusten bien a los datos.

Para evitar el elevado costo computacional de probar todas las combinaciones posibles de estos parámetros, usamos un algoritmo de búsqueda que puede retroceder cuando se aleja de la solución óptima. Este algoritmo utiliza técnicas para eliminar rápidamente configuraciones que no son válidas, como aquellas que resultan en poblaciones negativas. De esta manera, logramos reducir significativamente el tiempo de cómputo necesario.

El objetivo es minimizar la diferencia entre las simulaciones del modelo y los datos observados. Al implementar estas estrategias de búsqueda eficiente y validación, podemos encontrar los parámetros óptimos sin necesidad de evaluar exhaustivamente todas las posibilidades.

3. Introducción y motivación

En el estudio de la ecología y la biología de poblaciones, es muy importante predecir cómo crecerá una especie en un lugar determinado. Pero los modelos teóricos no siempre coinciden con la realidad. Para que coincidan, hay que ajustar o “calibrar” sus parámetros internos a las condiciones específicas del ecosistema.

La idea de este proyecto surge porque es necesario automatizar y optimizar este proceso de calibración. Si un biólogo tiene datos diarios sobre el crecimiento de una población, encontrar la tasa de crecimiento y la capacidad de carga del ecosistema a mano es un proceso lento y poco eficiente. Usar la fuerza bruta para explorar todas las combinaciones posibles garantiza encontrar la mejor respuesta, pero requiere muchos recursos computacionales cuando hay muchos datos.

Por eso, se propone usar la técnica de backtracking. Esta técnica construye soluciones de forma incremental y abandona las vías que ya no pueden ofrecer un mejor resultado, lo que reduce mucho el tiempo de ejecución.

4. Descripción del problema

Se cuenta con un registro empírico del tamaño de una población a lo largo de seis periodos (desde $t = 0$ hasta $t = 5$). Los datos observados son:

- Población inicial (x_0): 10 individuos.
- Datos observados (días 0 al 5): [10, 12, 15, 19, 24, 30]

Se busca ajustar estos datos a un modelo de crecimiento logístico discreto. Para ello, existen listas de candidatos posibles para dos parámetros cruciales:

- r (Tasa de crecimiento): [0,1,0,2,0,3,0,4]
- K (Capacidad de carga máxima): [50, 80, 100, 120]

El problema consiste en encontrar el par (r, K) exacto que produzca la curva de simulación con la menor desviación respecto a los datos observados por el biólogo. La desviación se mide mediante la suma de los errores absolutos en cada día.

5. Modelamiento con backtracking

Ecuación Matemática del Sistema

El núcleo del Eco Modelo es una ecuación de diferencia finita que representa el crecimiento logístico:

$$x_{t+1} = x_t + r \cdot x_t \cdot \left(1 - \frac{x_t}{K}\right)$$

Donde:

- x_t : población actual.
- x_{t+1} : población al día siguiente.
- r : factor de reproducción.
- K : límite máximo que soporta el ecosistema.

Estrategia de Backtracking y Poda

El árbol de búsqueda consiste en explorar cada combinación de (r, K) a lo largo del tiempo t . En lugar de simular los 5 días completos para cada una de las 16 combinaciones (lo que implicaría 80 pasos fijos), el algoritmo aplica dos reglas de poda restrictivas:

1. **Poda por Inviabilidad Biológica (Población negativa):** Si en el paso $t + 1$ la evaluación de la fórmula resulta en un $x_{t+1} < 0$, la configuración de (r, K) es descartada inmediatamente. Se interrumpe el ciclo y se retrocede (backtrack) al siguiente par de parámetros.
2. **Poda por Suboptimalidad (Error Alto):** El algoritmo almacena el `mejor_error_global`. A medida que avanza día a día para un par (r, K) , va sumando el error absoluto (la diferencia entre su cálculo y el dato real). Si en algún día intermedio, el `error_acumulado` iguala o supera el `mejor_error_global`, se concluye que este camino jamás será mejor que el que ya tenemos. El algoritmo aborta la simulación de los días restantes y retrocede.

Complejidad y Notación Landau (O Grande)

La notación O grande sirve para clasificar algoritmos según cómo crecen sus requerimientos de tiempo a medida que aumenta el tamaño de la entrada. Permite evaluar el "peor de los casos".

Sea N la cantidad de candidatos para r , M la cantidad de candidatos para K , y T la cantidad de días simulados.

- En **Fuerza Bruta**, la complejidad es estricta: $\mathcal{O}(N \cdot M \cdot T)$. El algoritmo siempre realizará todas las operaciones.
- En **Backtracking con Poda**, la complejidad en el peor de los casos sigue siendo $\mathcal{O}(N \cdot M \cdot T)$. Sin embargo, en el caso promedio o el mejor de los casos, la complejidad disminuye significativamente, acercándose a $\mathcal{O}(N \cdot M)$, ya que la mayoría de ramas internas dependientes de T son podadas tras las primeras evaluaciones.

6. Ejemplo manual pequeño

Para ilustrar el funcionamiento paso a paso, ejecutemos manualmente el algoritmo para la primera combinación candidata: $r = 0,1$ y $K = 50$.

- **Día 0:** Población inicial $x_0 = 10$. Dato real = 10 (Error base = 0).
- **Día 1:**
 - Fórmula: $x_1 = 10 + 0,1 \cdot 10 \cdot (1 - 10/50) = 10 + 1 \cdot 0,8 = 10,8$
 - Dato real: 12
 - Error del paso: $|10,8 - 12| = 1,2$
 - Error acumulado = 1,2
- **Día 2:**
 - Fórmula: $x_2 = 10,8 + 0,1 \cdot 10,8 \cdot (1 - 10,8/50) = 10,8 + 1,08 \cdot 0,784 = 11,6467$
 - Dato real: 15
 - Error del paso: $|11,6467 - 15| = 3,3533$
 - Error acumulado = $1,2 + 3,3533 = 4,5533$

En este punto, si el sistema tuviera previamente registrado un `mejor_error_global` de 4.0 derivado de otra rama anterior, el algoritmo detendría el cálculo del Día 3, Día 4 y Día 5 para esta combinación, ejecutando la poda y ahorrando esas iteraciones matemáticas.

7. Implementación en Python

La solución fue desarrollada en lenguaje Python, aprovechando su librería `matplotlib` para la visualización. La estructura del script consta de:

1. **Definición de hiperparámetros:** Inicialización de las listas de candidatos y los datos empíricos observados por el biólogo.
2. **Manejo de estado:** Variables que custodian el estado global de la búsqueda (`mejor_error_global`, `mejor_r`, `mejor_K`).
3. **Bucles anidados:** El núcleo del algoritmo, iterando sobre r y K , y un bucle interno sobre la secuencia temporal T .
4. **Lógica de Backtracking (Control de flujo):** Implementado mediante condicionales lógicos `if` y la instrucción `break`, que interrumpe el ciclo interior si se cumple alguna de las restricciones de poda.
5. **Módulo de visualización:** Una vez extraída la secuencia ganadora, se plotea un gráfico comparativo interpolando los puntos de la simulación ganadora versus el registro histórico real.

8. Pruebas y resultados

- **Calibración exitosa:** El script logra aislar una combinación exacta de r y K que presenta el menor error acumulativo posible dentro de las listas propuestas. El valor final del error absoluto permite cuantificar qué tan fidedigna es la curva predictiva respecto a la naturaleza empírica de la muestra.
- **Ahorro de recursos:** El algoritmo calcula e imprime un balance comparativo. Mientras que el método de fuerza bruta habría forzado las iteraciones completas T para cada par candidato, el registro de la variable `pasos_con_poda` evidencia una reducción notable en la cantidad de cálculos matemáticos. El porcentaje de "ahorro computacional" justifica la implementación de estructuras de decisión complejas dentro de ciclos de simulación, demostrando su eficiencia de escalabilidad.

9. Decisiones del grupo

- Se eligió la métrica de error que es el valor absoluto de la diferencia entre lo que se simuló y lo que es real. Esto se expresa como $|\textit{Simulado} - \textit{Real}|$. Esta opción es mejor que el error cuadrático porque muestra de manera clara y sencilla cuánto se desvía el modelo de la realidad. Además, esta métrica trata de la misma manera tanto los errores por defecto como los errores por exceso, sin dar demasiada importancia a las pequeñas variaciones.
- En cuanto al modelamiento, se decidió utilizar una ecuación en diferencias finitas. La razón es que los datos se recogen de manera discreta, es decir, se toman observaciones diarias y no se consideran fracciones de tiempo entre ellas.
- La estrategia de poda agresiva consiste en detener la evaluación en cuanto el error actual supere el mínimo histórico en cualquier momento. Esto garantiza que el proceso sea lo más eficiente posible. El razonamiento detrás de esto

es que, lógicamente, el error debería aumentar a medida que pasa el tiempo dentro de un ciclo de evaluación.

10. Conclusiones

- La implementación del algoritmo de backtracking fue muy efectiva para calibrar modelos biológicos. Esto se debe a que utiliza restricciones lógicas, conocidas como poda, para evitar un crecimiento explosivo en el número de combinaciones posibles al trabajar con listas largas de parámetros.
- Entender y aplicar la notación O Grande ayudó a demostrar teóricamente cómo este enfoque reduce el tiempo de ejecución. Aunque el límite superior de su complejidad sigue una curva similar a la de la fuerza bruta, en la práctica, el tiempo promedio de ejecución se reduce mucho.
- Esto hace que los algoritmos de backtracking sean ideales para investigaciones que requieren mucho procesamiento numérico y donde el tiempo de computación es crucial.

11. Referencias

1. Python Software Foundation. (2024). Python Language Reference. Obtenido de python.org.
2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press (Para los conceptos sobre Algoritmos de Backtracking y Notación de Landau).
4. Murray, J. D. (2002). *Mathematical Biology I: An Introduction*. Springer (Para las bases teóricas del crecimiento poblacional logístico).

12. Declaración de autoría y uso de herramientas

Declaramos que, los cálculos matemáticos y el flujo del informe son obra nuestra. Solo usamos herramientas de terceros como Google Colab como editor de texto, Matplotlib como librería de graficación, Asistente de IA (Gemini) el cual nos generó la base del código; y por último, nuestro uso de razonamiento para encontrar la lógica y funcionamiento del código generado por Gemini, el cual al saber que hace cada línea de código colocamos su respectivo comentario.